



# CE308 : โปรแกรมประยุกต์บนอุปกรณ์เคลื่อนที่

Week 10: | Redux

# Today's Class Agenda

- แนะนำ Redux
- หลักการทำงานของ Redux
- การติดตั้ง Redux และ Redux Toolkit
- ตัวอย่างการใช้งาน Redux
- Workshop



# 01

## แนะนำ Redux



# แนะนำ Redux

## Redux คืออะไร?

- Redux เป็น state management library ที่ได้รับความนิยมอย่างมากสำหรับการจัดการ state ในแอปพลิเคชัน React และ React Native
- Redux ทำงานบนแนวคิดของ Single Source of Truth (มีแหล่งข้อมูลหลักเพียงที่เดียว)
- Redux ใช้ Actions, Reducers, และ Store ในการจัดการข้อมูล
- Redux ไม่ได้ออกแบบมาเพื่อ React เท่านั้น แต่ใช้ได้กับหลายแพลตฟอร์ม รวมถึง React Native



# ปัญหาที่ Redux เข้ามาแก้ไข

## ปัญหาที่ Redux เข้ามาแก้ไข

### 1. State ที่กระจายอยู่ตาม Components

- ใน React ปกติ state จะถูกเก็บไว้ใน Component และส่งลงไปยังลูกผ่าน Props
- เมื่อแอปเติบโตขึ้น State อาจต้องแชร์ระหว่าง Components หลายตัว ทำให้โค้ดซับซ้อน

### 2. Prop Drilling

- เมื่อ Components ลึกลงไปในโครงสร้างต้องใช้ข้อมูลจาก State ที่อยู่ข้างบน
- จำเป็นต้องส่ง Props ผ่าน Components หลายชั้นโดยที่บางตัวไม่ใช่ข้อมูลเลย

### 3. Context API อาจไม่เพียงพอ

- React มี Context API ซึ่งช่วยให้แชร์ข้อมูลระหว่าง Components ได้ แต่เมื่อ State มีความซับซ้อนมากขึ้น การจัดการ Context API อาจทำให้โค้ดอ่านยาก

## Redux เข้ามาช่วยจัดการปัญหานี้อย่างไร?

- Redux ใช้ Single Source of Truth (แหล่งข้อมูลเพียงที่เดียว)
- ข้อมูลทั้งหมดจะถูกจัดเก็บไว้ใน Store และทุก Components สามารถเข้าถึงข้อมูลได้โดยตรง
- ใช้ Actions และ Reducers เพื่ออัปเดตข้อมูลอย่างมีระบบ



# 02

## หลักการทำงานของ Redux



# หลักการทำงานของ Redux

## Redux ทำงานอย่างไร?

Redux ทำงานบนแนวคิดหลัก 3 อย่าง:

- Single Source of Truth
  - State ทั้งหมดของแอปถูกเก็บไว้ในที่เดียวคือ "Store"
  - ทำให้การจัดการข้อมูลเป็นระเบียบ ลดความซับซ้อน
- State เป็น Read-Only
  - State ไม่สามารถเปลี่ยนแปลงโดยตรงได้ ต้องใช้ Actions และ Reducers
  - ทำให้การเปลี่ยนแปลง State มีความชัดเจนและสามารถตรวจสอบย้อนกลับได้
- Changes ผ่าน Pure Functions
  - Reducers เป็นฟังก์ชันที่กำหนดว่า State จะเปลี่ยนแปลงอย่างไรเมื่อได้รับ Action
  - Reducers ต้องเป็น Pure Functions (ฟังก์ชันที่ให้ผลลัพธ์เดียวกันเสมอเมื่อได้รับ input เดียวกัน)

# State

- State: คือข้อมูลทั้งหมดที่เกี่ยวข้องกับแอป เช่น
  - ข้อมูลของผู้ใช้ (ชื่อ, อีเมล)
  - รายการสินค้าในตะกร้าสินค้า
  - ค่าตัวเลขในตัวนับ (Counter)
- ตัวอย่าง State สำหรับแอปตัวนับ (Counter)

```
const initialState = {  
  value: 0  
};
```



# Actions

- Actions: เป็นคำสั่งที่บอกให้ Redux เปลี่ยนแปลงข้อมูล
  - ตัวอย่าง Action มีลักษณะเป็น Object ที่มี type และ payload (ถ้ามี)

```
const incrementAction = { type: 'counter/increment' };  
const decrementAction = { type: 'counter/decrement' };  
const incrementByAmountAction = { type: 'counter/incrementByAmount', payload: 5 };
```

# Reducers

- Reducer เป็นแนวคิดที่มาจาก Redux ซึ่งเป็นไลบรารีสำหรับการจัดการ state ในแอป React และ React Native แบบรวมศูนย์ (centralized state management) โดย reducer function จะเป็นตัวที่กำหนดว่าควรเปลี่ยนแปลง state อย่างไรเมื่อได้รับ action ใหม่
- Reducer ใช้หลักการของ Pure Function ซึ่งหมายความว่า
  - ให้ค่าเดียวกันเมื่อได้รับ input เดียวกัน
  - ไม่มีผลข้างเคียง (side effects)
  - ไม่เปลี่ยนแปลงค่าเดิม (immutable state)

# ตัวอย่าง Reducer

```
import { createSlice, PayloadAction } from '@reduxjs/toolkit';

// 1 กำหนดรูปแบบของ State
interface CounterState {
  value: number;
}

// 2 กำหนดค่าเริ่มต้นของ State
const initialState: CounterState = { value: 0 };
```

```
// 3 ใช้ createSlice สร้าง Reducer และ Actions
const counterslice = createSlice({
  name: 'counter', // ชื่อ Slice นี้
  initialState, // State เริ่มต้น
  reducers: {
    // Action: เพิ่มค่า
    increment: (state) => {
      state.value += 1;
    },
    // Action: ลดค่า
    decrement: (state) => {
      state.value -= 1;
    },
    // Action: เพิ่มค่าตามจำนวนที่กำหนด
    incrementByAmount: (state, action: PayloadAction<number>) => {
      state.value += action.payload;
    },
  },
});

// 4 Export Actions เพื่อให้ Component สามารถเรียกใช้ได้
export const { increment, decrement, incrementByAmount } = counterslice.actions;

// 5 Export Reducer เพื่อนำไปใช้ใน Store
export default counterslice.reducer;
```

# Store

- Store เป็นที่เก็บ State ทั้งหมดของแอป และเป็นที่ยอมรับของ Reducers
- ตัวอย่างการสร้าง Store

```
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';

// 1 สร้าง Store และกำหนด Reducer
export const store = configureStore({
  reducer: {
    counter: counterReducer, // ใช้ counterSlice ที่สร้างไว้
  },
});

// 2 กำหนด Type ของ RootState และ AppDispatch
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

- RootState → ใช้เพื่อให้ TypeScript รู้จักโครงสร้างของ State
- AppDispatch → ใช้เพื่อให้ TypeScript ตรวจสอบค่าที่ส่งผ่าน dispatch ได้ถูกต้อง

# Dispatch

- Dispatch เป็นฟังก์ชันที่ใช้ส่ง Action เพื่อให้ Redux เปลี่ยนแปลง State
- ตัวอย่างการใช้ dispatch

```
import { useDispatch } from 'react-redux';
import { increment, decrement } from '../redux/counterSlice';

const CounterScreen = () => {
  const dispatch = useDispatch();

  return (
    <View>
      <Button title="Increment" onPress={() => dispatch(increment())} />
      <Button title="Decrement" onPress={() => dispatch(decrement())} />
    </View>
  );
};
```

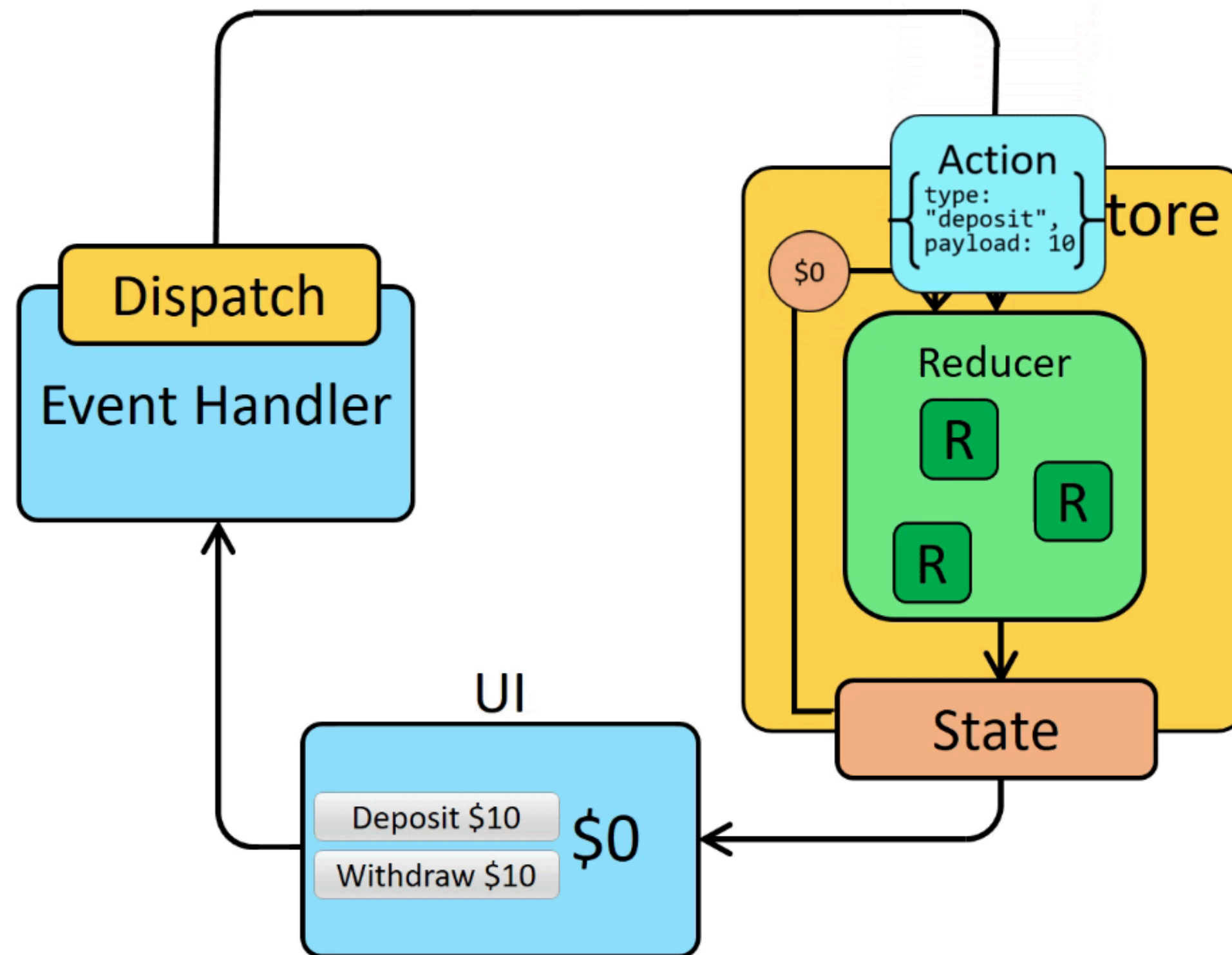


# สรุป: Redux ทำงานอย่างไร?

Redux ทำงานเป็นวงจรตามลำดับนี้:

1. ผู้ใช้กระทำบางอย่าง เช่น กดปุ่มเพิ่มจำนวน (Counter +1)
2. Component เรียกใช้ Dispatch เพื่อส่ง Action ไปยัง Redux
3. Redux นำ Action ไปที่ Reducer เพื่อตรวจสอบว่าต้องเปลี่ยนแปลง State อย่างไร
4. Reducer คำนวณ State ใหม่ แล้ว Redux อัปเดต Store
5. Component ที่ใช้ State นั้นจะ Re-render เพื่อแสดงผลข้อมูลใหม่

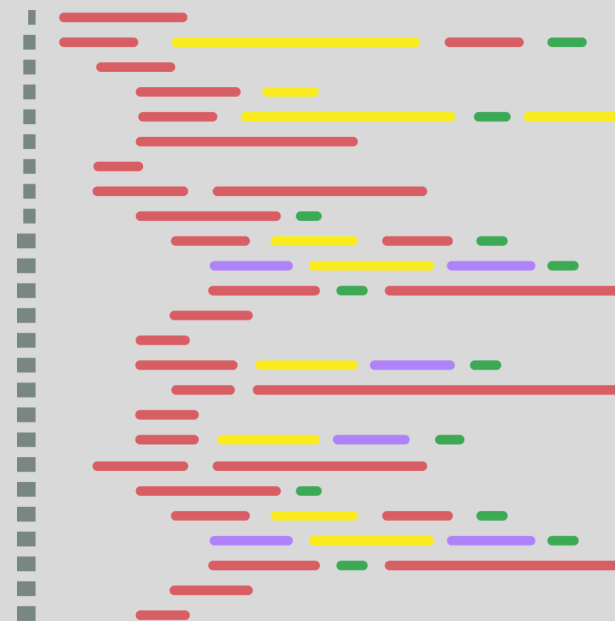
# ภาพรวม Redux



<https://redux.js.org/tutorials/fundamentals/part-2-concepts-data-flow>

# 03

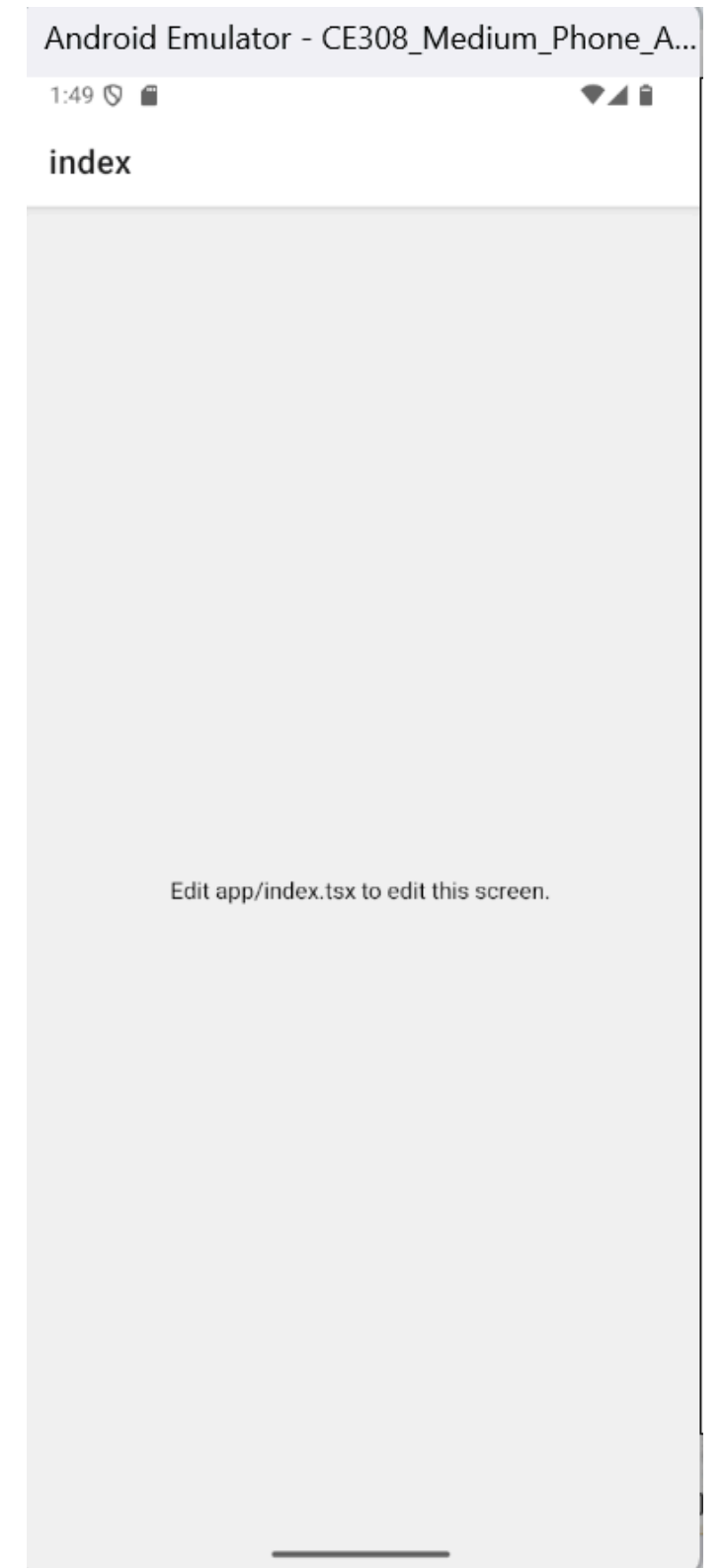
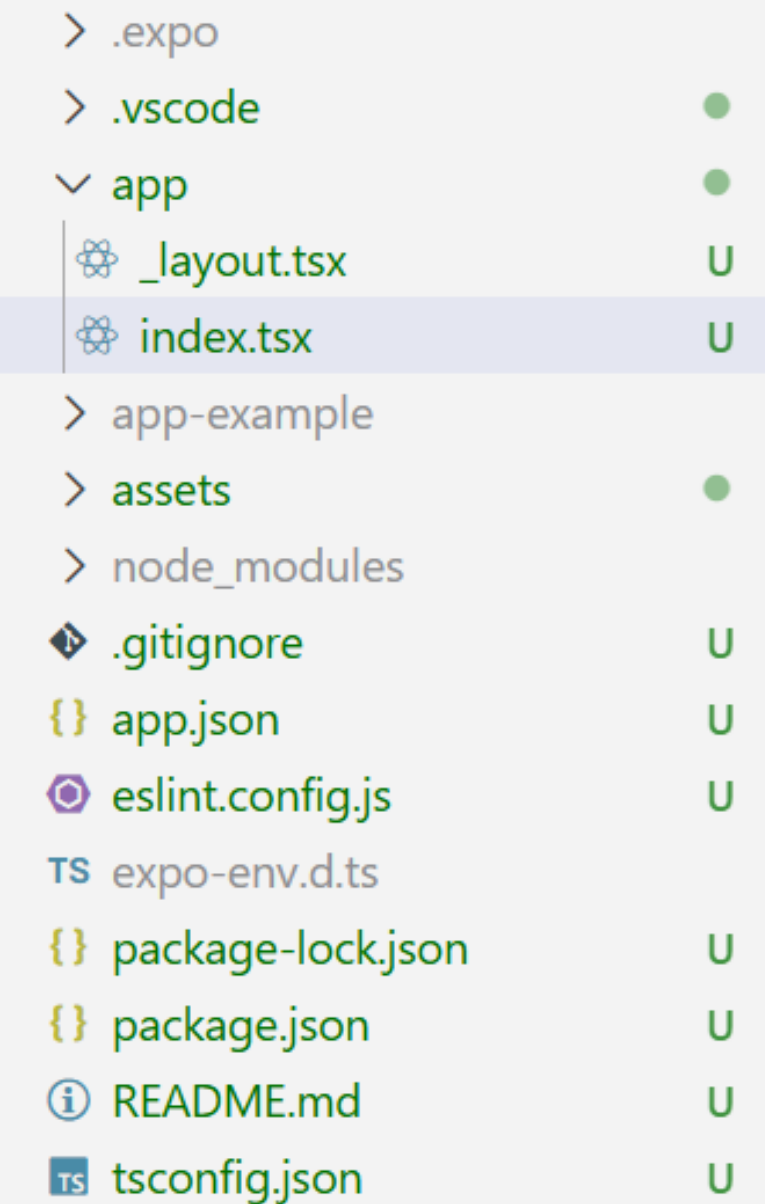
## การติดตั้ง Redux และ Redux Toolkit





# ติดตั้งและรีเซตค่า

1. ใช้คำสั่ง **`npx create-expo-app@latest`**
2. กำหนดชื่อโปรเจกต์
3. ใช้คำสั่ง **`cd`** เข้าตามชื่อโปรเจกต์
4. ใช้คำสั่ง **`npm run reset-project`**
5. รันโปรเจกต์โดยใช้คำสั่ง **`npm run android`**



# การติดตั้ง Redux และ Redux Toolkit

- ติดตั้ง Redux และ Redux Toolkit ด้วยคำสั่ง `npm install @reduxjs/toolkit react-redux`

```
added 7 packages, and audited 1177 packages in 5s
```

```
110 packages are looking for funding  
  run `npm fund` for details
```

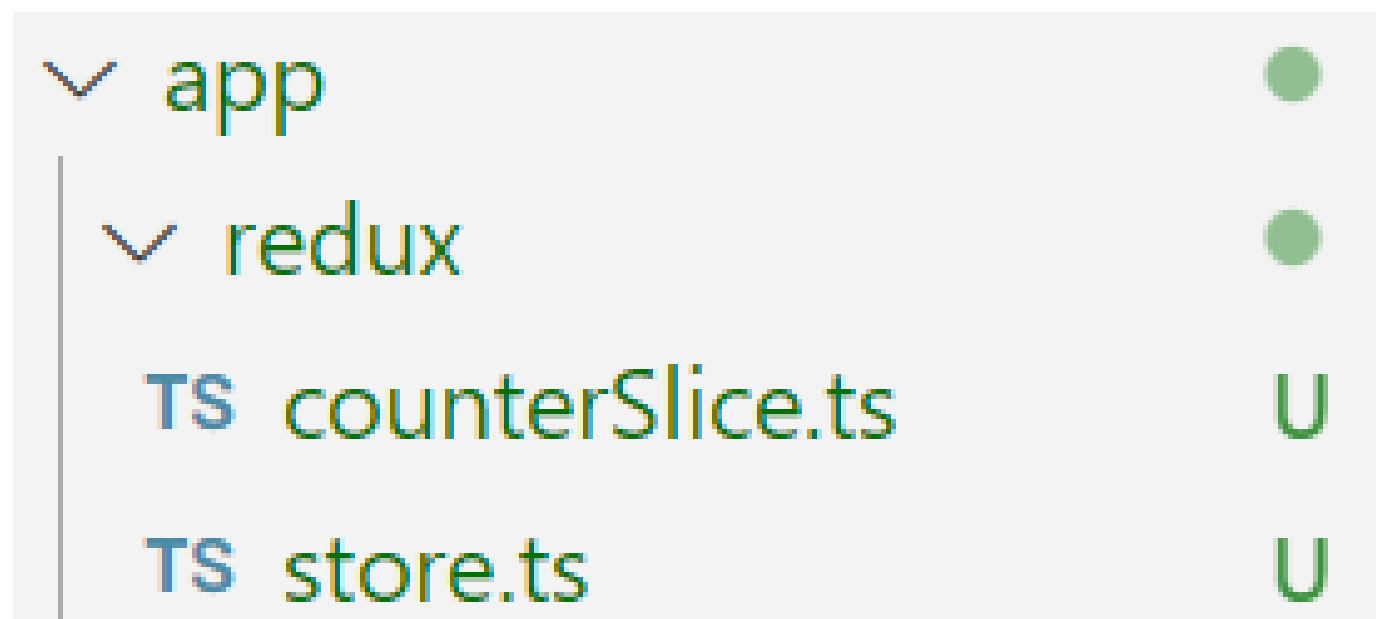
```
5 low severity vulnerabilities
```

```
To address all issues (including breaking changes), run:  
  npm audit fix --force
```

```
Run `npm audit` for details.
```

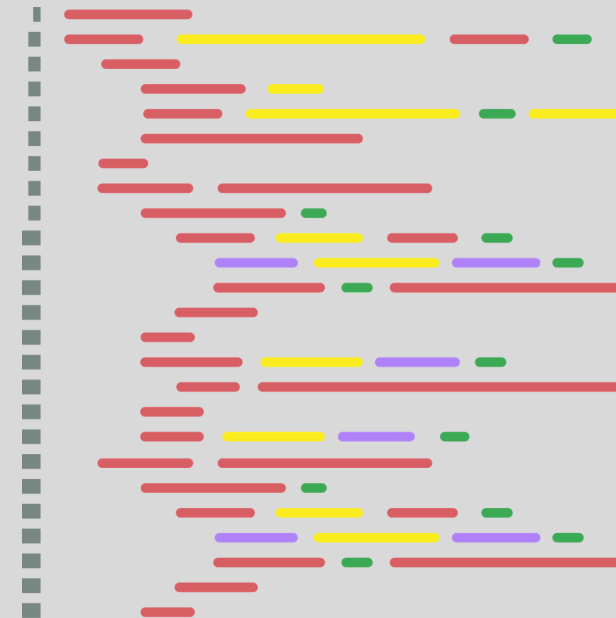
# การสร้างไฟล์ Store และ Reducer

- สร้างไฟล์เตอร์ redux อยู่ในโฟลเดอร์ app
- สร้างไฟล์
  - Store ชื่อ store.ts
  - Reducer ชื่อ counterSlice.ts



# 04

## ตัวอย่างการใช้งาน Redux



# แก้ไขไฟล์ counterSlice.ts

- แก้ไขหรือสร้างไฟล์ counterSlice.ts

app > redux > TS counterSlice.ts > [🔗] counterSlice > 🔗 reducers > 📦 increment

```
1  import { createSlice, PayloadAction } from '@reduxjs/toolkit';
2
3  // 1 กำหนดรูปแบบของ State
4  interface CounterState {
5    | value: number;
6  }
7  // 2 กำหนดค่าเริ่มต้นของ State
8  const initialState: CounterState = { value: 0 };
```

```
10 // 3 ใช้ createSlice สร้าง Reducer และ Actions
11 const counterSlice = createSlice({
12   name: 'counter', // ชื่อ Slice นี้
13   initialState, // State เริ่มต้น
14   reducers: {
15     // Action: เพิ่มค่า
16     increment: (state) => {
17       | state.value += 1;
18     },
19     // Action: ลดค่า
20     decrement: (state) => {
21       | state.value -= 1;
22     },
23     // Action: เพิ่มค่าตามจำนวนที่กำหนด
24     incrementByAmount: (state, action: PayloadAction<number>) => {
25       | state.value += action.payload;
26     },
27   },
28 });
```

```
30 // 4 Export Actions เพื่อให้ Component สามารถเรียกใช้ได้
31 export const { increment, decrement, incrementByAmount } = counterSlice.actions;
32
33 // 5 Export Reducer เพื่อนำไปใช้ใน Store
34 export default counterSlice.reducer;
```



# configureStore

- แก้ไขไฟล์ store.ts เพื่อสร้าง Store และ กำหนด Reducer

app > redux > TS store.ts > ...

```
1  import { configureStore } from '@reduxjs/toolkit';
2  import counterReducer from './counterSlice';
3
4  // 1 สร้าง Store และกำหนด Reducer
5  export const store = configureStore({
6    reducer: {
7      counter: counterReducer, // ใช้ counterSlice ที่สร้างไว้
8    },
9  });
10
11 // 2 กำหนด Type ของ RootState และ AppDispatch
12 // ใช้เพื่อให้ TypeScript รู้จักโครงสร้างของ State
13 export type RootState = ReturnType<typeof store.getState>;
14 // ใช้เพื่อให้ TypeScript ตรวจสอบค่าที่ส่งผ่าน dispatch ได้ถูกต้อง
15 export type AppDispatch = typeof store.dispatch;
```

# สร้างไฟล์แสดงผล

- สร้างไฟล์เตอร์ screens
- สร้างไฟล์ CounterScreen.tsx

app > screens >  CounterScreen.tsx > ...

```
1  import React from 'react';
2  import { View, Text, Button, StyleSheet } from 'react-native';
3  import { useSelector, useDispatch } from 'react-redux';
4  import { RootState, AppDispatch } from '../redux/store';
5  import { increment, decrement, incrementByAmount } from '../redux/counterSlice';
6
7  const CounterScreen = () => {
8    // 1 ใช้ useSelector ดึงค่า State
9    const count = useSelector((state: RootState) => state.counter.value);
10   // 2 ใช้ useDispatch เพื่อส่ง Actions
11   const dispatch = useDispatch<AppDispatch>();
12
13   return (
14     <View style={styles.container}>
15       <Text style={styles.countText}>Count: {count}</Text>
16       <Button title="Increment" onPress={() => dispatch(increment())} />
17       <Button title="Decrement" onPress={() => dispatch(decrement())} />
18       <Button title="Increment by 5" onPress={() => dispatch(incrementByAmount(5))} />
19     </View>
20   );
21 };
```

# สร้างไฟล์แสดงผล (ต่อ)

```
23  // 3 สไตล์ของ Component
24  const styles = StyleSheet.create({
25    container: {
26      flex: 1,
27      justifyContent: 'center',
28      alignItems: 'center',
29    },
30    countText: {
31      fontSize: 24,
32      marginBottom: 20,
33    },
34  });
35
36  export default CounterScreen;
```

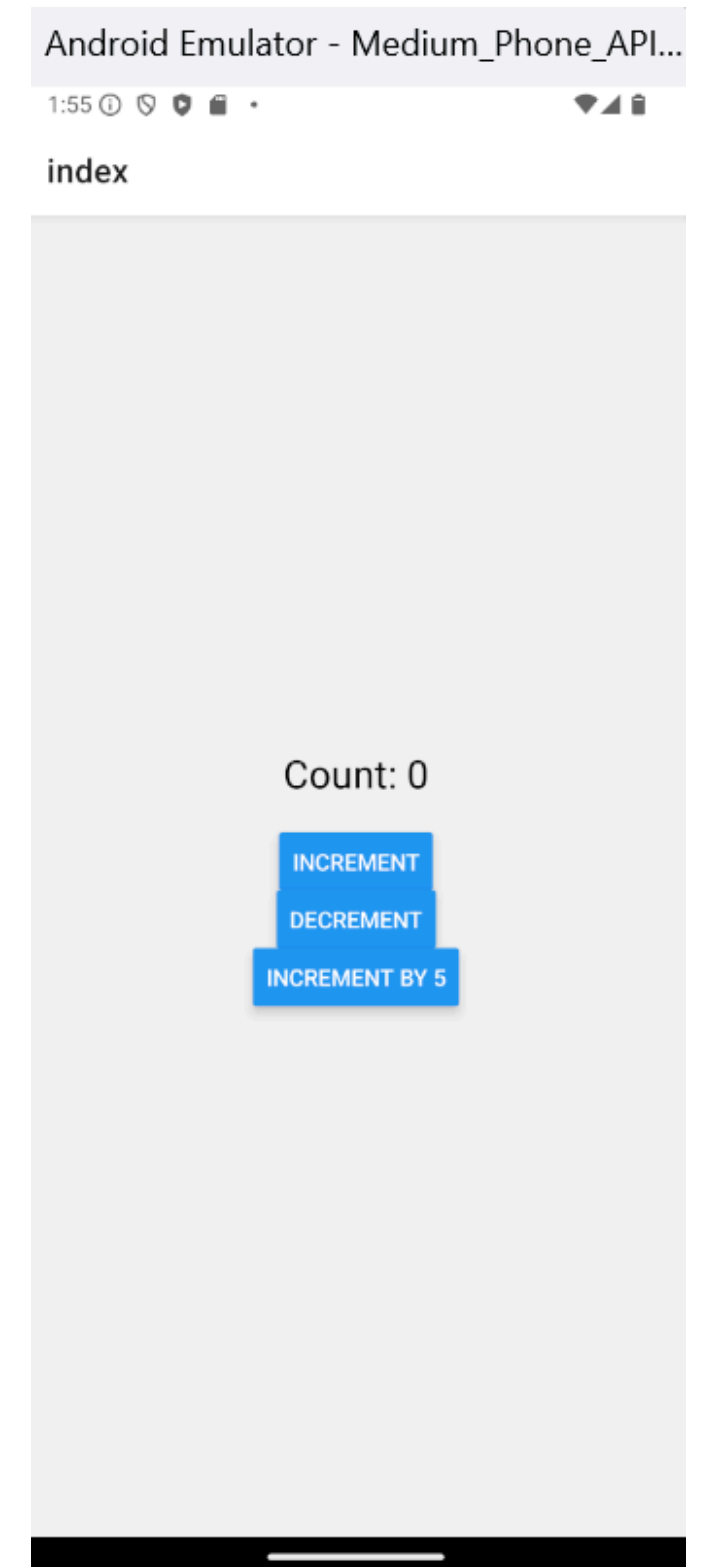


# แก้ไขไฟล์ index.tsx

- ไฟล์ index.tsx (เพิ่ม Redux Provider)
  - ใช้ Provider ครอบแอปทั้งหมด เพื่อให้ทุก Component สามารถเข้าถึง Redux Store ได้
- store={store}
  - ส่ง Store ที่สร้างไว้ให้ Provider เพื่อให้แอปรู้จัก Redux

app > index.tsx > ...

```
1  import React from 'react';
2  import { Provider } from 'react-redux';
3  import { store } from '../redux/store';
4  import CounterScreen from '../screens/CounterScreen';
5
6  export default function App() {
7    return (
8      <Provider store={store}>
9        <CounterScreen />
10     </Provider>
11   );
12 }
```







# Workshop 10

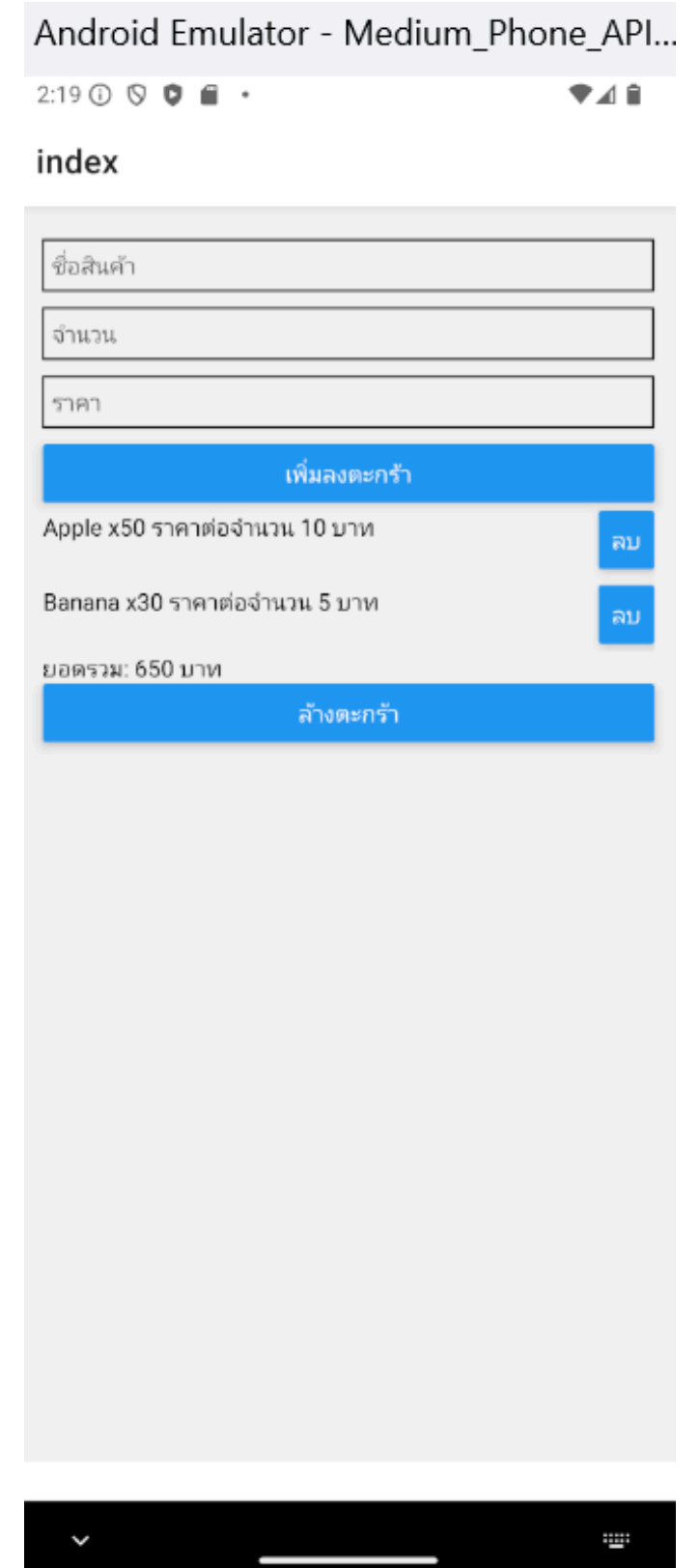


# 10.1 ระบบตะกร้าสินค้า (Shopping Cart)

สร้างแอปที่สามารถ เพิ่มสินค้าไปยังตะกร้า, ลบสินค้าออก, และแสดงยอดรวมราคา  
ฟีเจอร์ที่ต้องมี

- กรอกข้อมูลสินค้า (ชื่อ, จำนวน, ราคา) แล้วกด "เพิ่มลงตะกร้า"
- แสดงรายการสินค้าทั้งหมดที่อยู่ในตะกร้า
- สามารถลบสินค้าที่ไม่ต้องการออกจากตะกร้าได้
- แสดงยอดรวมราคาสินค้าในตะกร้า
- Actions ที่ต้องมี:
  - addItem(payload: CartItem) → เพิ่มสินค้าไปยังตะกร้า
  - removeItem(payload: string) → ลบสินค้าออกจากตะกร้า
  - clearCart() → ล้างตะกร้าสินค้า
- Redux State ที่ต้องใช้

```
interface CartItem {  
  id: string;  
  name: string;  
  quantity: number;  
  price: number;  
}  
  
interface CartState {  
  items: CartItem[];  
  totalAmount: number;  
}
```



# 10.2 ระบบ To-Do List พร้อมสถานะ

สร้างแอป To-Do List ที่สามารถเพิ่มงานใหม่, เปลี่ยนสถานะว่างานเสร็จหรือยัง, และลบงานที่ไม่ต้องการ  
ฟีเจอร์ที่ต้องมี

- เพิ่มงานใหม่เข้าไปในรายการ
- กดเปลี่ยนสถานะ (ยังไม่เสร็จ → เสร็จแล้ว ) หากเสร็จแล้วให้มีขีดที่ชื่องาน
- ลบงานที่ไม่ต้องการออก
- แสดงจำนวนงานทั้งหมด
- Actions ที่ต้องมี:
  - addTodo(payload: Todo) → เพิ่มงาน
  - toggleTodo(payload: string) → เปลี่ยนสถานะเสร็จ/ไม่เสร็จ
  - removeTodo(payload: string) → ลบงาน
- Redux State ที่ต้องใช้

```
interface Todo {  
  id: string;  
  text: string;  
  completed: boolean;  
}  
  
interface TodoState {  
  todos: Todo[];  
}
```

